```
import numpy as np

states = ["S0", "S1", "Wall", "G"]
non_terminal_states = ["S0", "S1"]
actions = ["Up", "Down", "Left", "Right"]
gamma = 0.9
theta = 1e-4

transitions = {
    "S0": {
        "Up": ("S0", -1),
        "Down": ("S1", -1),
        "Left": ("S0", -1),
        "Right": ("S0", -1),
    },
    "S1": {
        "Up": ("S0", -1),
        "Down": ("S1", -1),
        "Left": ("S1", -1),
        "Right": ("G", 10),
    },
}

print("=== TASK 1: MDP INSPECTION ===")
print(f"Non-Terminal States: {non_terminal_states}")
print(f"Actions: {actions}")
print(f"Discount Factor (gamma): {gamma}")
print("Transition Rules Defined Successfully.\n")

print("=== TASK 2: VERIFY TRANSITIONS ===")
for state in non_terminal_states:
    for action in actions:
        next_s, reward = transitions[state][action]
        print(
            f"State: {state} | Action: {action:<5} -> Next State: {next_s} | Reward: {reward}"
        )
print()

V = {s: 0.0 for s in non_terminal_states}
V["G"] = 0.0

print("=== TASK 3: INITIAL STATE VALUES ===")
for s, val in V.items():
    print(f"V({s}) = {val:.2f}")
print()

print("=== TASK 4 & 5: VALUE ITERATION LOG ===")
iteration = 0
while True:
```

```
    iteration += 1
    V_new = V.copy()
    delta = 0.0

    for state in non_terminal_states:
        q_values = []
        for action in actions:
            next_state, reward = transitions[state][action]
            q_val = reward + gamma * V[next_state]
            q_values.append(q_val)

        v_max = max(q_values)
        delta = max(delta, abs(v_max - V[state]))
        V_new[state] = v_max

    V = V_new.copy()
    print(
        f"Iteration {iteration:2d} | V(S0) = {V['S0']:6.2f} | V(S1) = {V['S1']:6.2f} | Delta = {delta:6.4f}"
    )

    if delta < theta:
        break

print(f"\nValue Iteration Converged in {iteration} Iterations.\n")

print("=== TASK 6: FINAL ACTION VALUES Q*(s, a) ===")
Q = {s: {} for s in non_terminal_states}
for state in non_terminal_states:
    print(f"\nState {state}:")
    for action in actions:
        next_state, reward = transitions[state][action]
        q_val = reward + gamma * V[next_state]
        Q[state][action] = q_val
        print(f"  Q*({state}, {action:<5}) = {q_val:6.2f}")
print()

print("=== TASK 7: OPTIMAL POLICY ===")
optimal_policy = {}
for state in non_terminal_states:
    max_q = max(Q[state].values())
    best_actions = [a for a, q in Q[state].items() if np.isclose(q, max_q)]
    optimal_policy[state] = best_actions
    print(f"Optimal Action for {state}: {best_actions}")
print()

print("=== TASK 8: BELLMAN OPTIMALITY VERIFICATION ===")
for state in non_terminal_states:
    max_q = max(Q[state].values())
    v_val = V[state]
    print(
        f"State {state}: V*({state}) = {v_val:.2f} | max_a Q*({state}, a) = {max_q:.2f} -> Match: {np.isclose(v_val, max_q)}"
    )
```

```
print()

print("=== TASK 9: RESULT EXPLANATION ===")
explanation = """
1. Optimal Actions:
   - For S0: The optimal action is 'Down' because moving to S1 places the agent 1 step away from the Goal G.
   - For S1: The optimal action is 'Right' because it directly leads to Goal G with high positive reward (+10).

2. Impact of Goal Reward (+10):
   - The large positive goal reward creates a strong value gradient across the grid.
   - Even though each move costs -1, reaching Goal G yields enough net gain (+10) to pull the values of adjacent states up.

3. Role of Discount Factor (gamma = 0.9):
   - Gamma ensures future rewards are discounted proportionally to step distance.
   - S1 is 1 step away from G, yielding V*(S1) = 10.
   - S0 is 2 steps away from G, yielding V*(S0) = -1 + 0.9(10) = 8.0.
"""
print(explanation)
```

```
=== TASK 1: MDP INSPECTION ===
Non-Terminal States: ['S0', 'S1']
Actions: ['Up', 'Down', 'Left', 'Right']
Discount Factor (gamma): 0.9
Transition Rules Defined Successfully.

=== TASK 2: VERIFY TRANSITIONS ===
State: S0 | Action: Up    -> Next State: S0 | Reward: -1
State: S0 | Action: Down  -> Next State: S1 | Reward: -1
State: S0 | Action: Left  -> Next State: S0 | Reward: -1
State: S0 | Action: Right -> Next State: S0 | Reward: -1
State: S1 | Action: Up    -> Next State: S0 | Reward: -1
State: S1 | Action: Down  -> Next State: S1 | Reward: -1
State: S1 | Action: Left  -> Next State: S1 | Reward: -1
State: S1 | Action: Right -> Next State: G | Reward: 10

=== TASK 3: INITIAL STATE VALUES ===
V(S0) = 0.00
V(S1) = 0.00
V(G) = 0.00

=== TASK 4 & 5: VALUE ITERATION LOG ===
Iteration  1 | V(S0) =  -1.00 | V(S1) =  10.00 | Delta = 10.0000
Iteration  2 | V(S0) =   8.00 | V(S1) =  10.00 | Delta = 9.0000
Iteration  3 | V(S0) =   8.00 | V(S1) =  10.00 | Delta = 0.0000

Value Iteration Converged in 3 Iterations.

=== TASK 6: FINAL ACTION VALUES Q*(s, a) ===

State S0:
  Q*(S0, Up   ) =   6.20
  Q*(S0, Down ) =   8.00
  Q*(S0, Left ) =   6.20
  Q*(S0, Right) =   6.20

State S1:
  Q*(S1, Up   ) =   6.20
```

```
  Q*(S1, Down ) =   8.00
  Q*(S1, Left ) =   8.00
  Q*(S1, Right) =  10.00

=== TASK 7: OPTIMAL POLICY ===
Optimal Action for S0: ['Down']
Optimal Action for S1: ['Right']

=== TASK 8: BELLMAN OPTIMALITY VERIFICATION ===
State S0: V*(S0) = 8.00 | max_a Q*(S0, a) = 8.00 -> Match: True
State S1: V*(S1) = 10.00 | max_a Q*(S1, a) = 10.00 -> Match: True

=== TASK 9: RESULT EXPLANATION ===

1. Optimal Actions:
   - For S0: The optimal action is 'Down' because moving to S1 places the agent 1 step away from the Goal G.
   - For S1: The optimal action is 'Right' because it directly leads to Goal G with high positive reward (+10).

2. Impact of Goal Reward (+10):
```